

Implementation and evaluation of the reinforcement learning methods Self-Play and League Training in the MicroRTS environment

Niklas Glaser

Heinrich-Heine-Universität Düsseldorf

April 2026

Goals and Research Question

Goals

- improve the Base Agent with League Training (LT)
- training against a more diverse set of opponents
- focus on robustness and generalization

Context: Base Agent

- combination of Behavioral Cloning (BC) and Proximal Policy Optimization (PPO)
- strong baseline, but weaknesses against several opponents

Research Questions

- How much does LT with PPO improve robustness and generalization of the BC/PPO-pretrained agent?
- How strongly does this depend on the diversity of initial agents and BC experts?

Setting

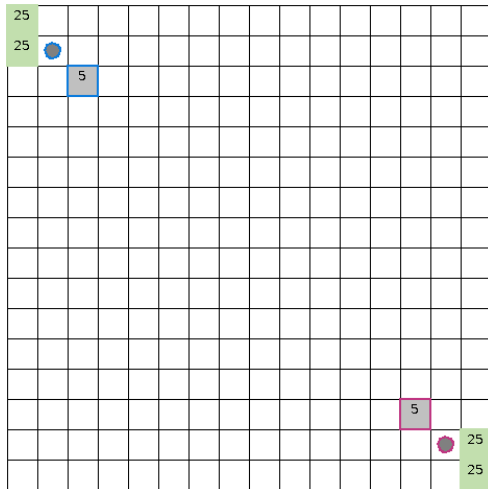
MicroRTS

- simplified RTS environment for RL research
- still contains resource management, unit production and combat
- supports controlled and repeatable experiments

Main RL challenges

- large combinatorial action space
- simultaneous decisions
- delayed and sparse rewards
- robustness against diverse opponents

► [Open MicroRTS Example Video](#)



Training Pipeline



BC and BC-FT

- imitate experts
- BC-FT: only winning replays

PPO

- iteratively update policy
- refine the policy through exploration

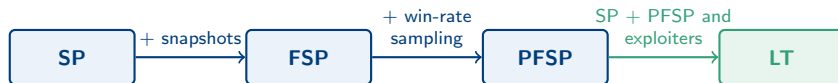
LT

- train against an evolving opponent population
- add exploiters

PPO fine-tuning

- repair the main remaining weakness after LT
- add WorkerRushAI as opponent

From SP to LT



cyclical training

A

B

C

A, B and C are policies

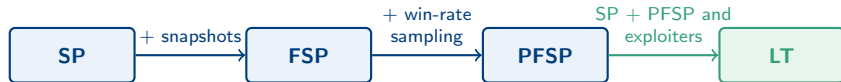
Step shown in the cycle

- A, B and C: example policies

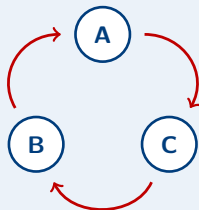
Why LT helps

- improves stability by identifying weaknesses
- creates diverse opponents through exploiters

From SP to LT



cyclical training



Red: policy strength
Blue: possible SP cycle

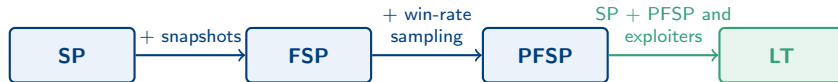
Step shown in the cycle

- $B > A, C > B, A > C$

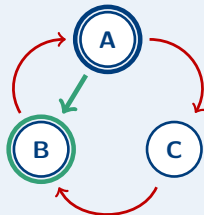
Why LT helps

- improves stability by identifying weaknesses
- creates diverse opponents through exploiters

From SP to LT



cyclical training



Red: policy strength
Blue: possible SP cycle

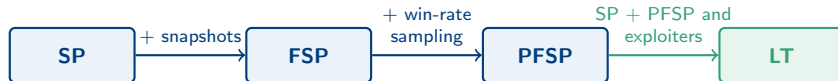
Step shown in the cycle

- FSP starts from A
- likely trains toward B

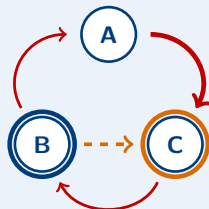
Why LT helps

- improves stability by identifying weaknesses
- creates diverse opponents through exploiters

From SP to LT



cyclical training



Red: policy strength
Blue: possible SP cycle

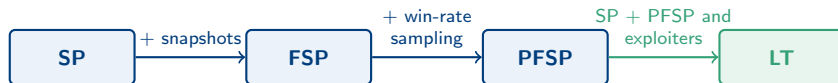
Step shown in the cycle

- FSP opponents: A and B
- A makes C less rewarding
- cycle less likely

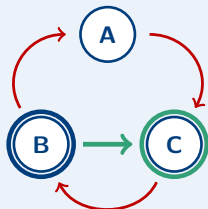
Why LT helps

- improves stability by identifying weaknesses
- creates diverse opponents through exploiters

From SP to LT



cyclical training



Red: policy strength
Blue: possible SP cycle

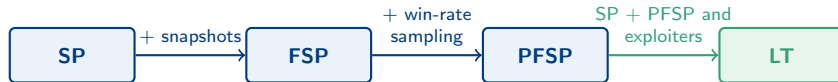
Step shown in the cycle

- in case of SP
- SP current policy: B
- likely trains toward C

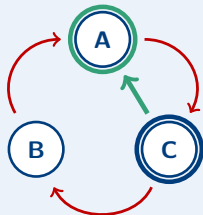
Why LT helps

- improves stability by identifying weaknesses
- creates diverse opponents through exploiters

From SP to LT



cyclical training



Red: policy strength
Blue: possible SP cycle

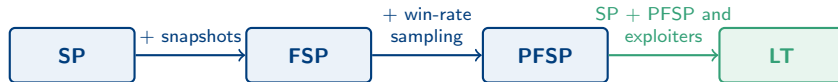
Step shown in the cycle

- SP current policy: C
- likely trains toward A

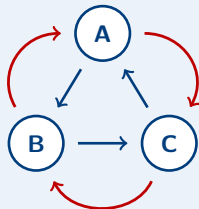
Why LT helps

- improves stability by identifying weaknesses
- creates diverse opponents through exploiters

From SP to LT



cyclical training



Red: policy strength
Blue: possible SP cycle

Step shown in the cycle

- training cycle can repeat
- $A \rightarrow B \rightarrow C \rightarrow A \rightarrow \dots$

Why LT helps

- improves stability by identifying weaknesses
- creates diverse opponents through exploiters

Main Agent (MA)

- primary policy
- resulting agent

Historical Agents (HA)

- frozen checkpoints kept in the opponent pool
- created by all other league agent types

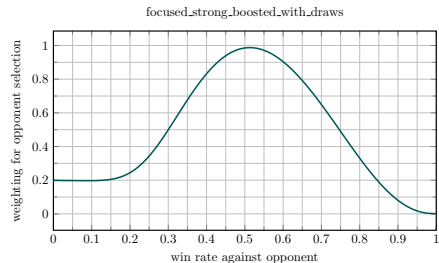
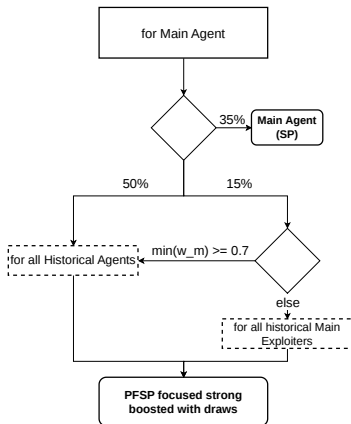
Main Exploiters (ME)

- exploit weaknesses of the MA
- reset when checkpointed

League Exploiters (LE)

- exploit broader weaknesses across the league
- can reset when checkpointed

LT MA Matchmaking

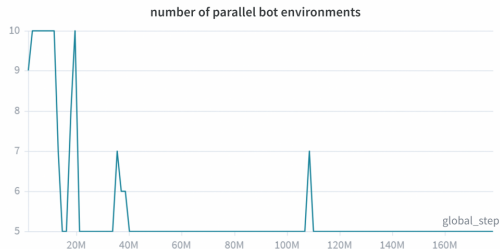


- 35% self-play
- 50% PFSP over HAs
- 15% PFSP over HMEs if win rate is low
- PFSP prioritizes unresolved matchups

Bot Environments

Bot environments

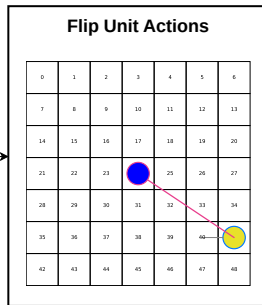
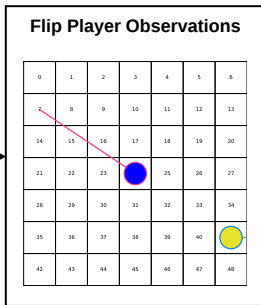
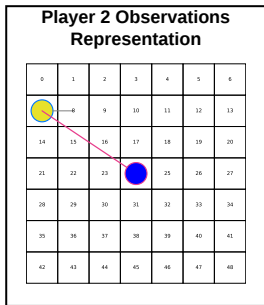
- additional dynamic bot environments (CoacAI and Mayari)
- scripted bots: CoacAI, Mayari, WorkerRushAI, PassiveAI, LightRushAI, RandomAI and RandomBiasedAI
- MCTS based bots: Rojo, MixedBot, Izanagi, Tiamat, Droplet, GuidedRojoA3N and NaiveMCTS



Observation Adjustment for SP

Observation adjustment

- Base Agent only trained as Player 1
- flip Player-2 GridNet observations
- flip masks and action directions



Implementation Changes

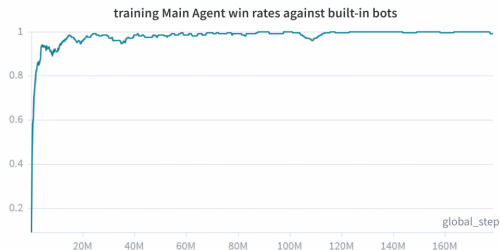
Key changes

- MicroRTS score: denser signal in sparse-reward games
- annealed entropy coefficient: more exploration early, less later
- new initial agents: new start points for exploiters

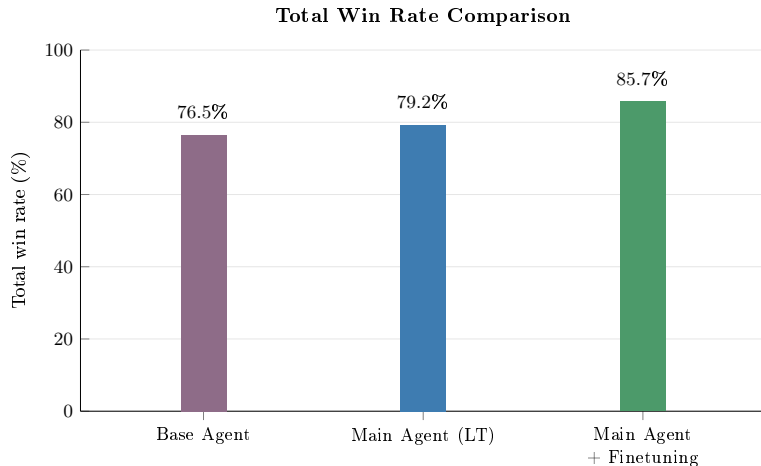
Experiment Setup

Setup

- 1 map: basesWorkers16x16A
- 1 MA, 4 ME, 1 LE
- for each 25 environments
- 5 to 10 dynamic bot environments
- ~130M LT steps, ~100 h
- max: 160 parallel environments
- eval: at least 100 games vs 14 opponents



Main Quantitative Result



Loss rate

- loss rate: **7.7% → 2.2%**
- different Base Agent version than in the prior work

Evaluation Results (Win Rate)

Opponent	Base	LT	LT + FT
CoacAI	96.0	100.0	100.0
Mayari	67.3	100.0	100.0
WorkerRushAI	100.0	35.0	99.5
RandomBiasedAI	79.0	97.0	97.5
MixedBot	43.3	57.0	60.5
GuidedRojoA3N	74.7	87.0	88.5
Tiamat	93.0	99.0	99.0
Droplet	67.7	62.0	75.5
overall	76.5	79.2	85.7

How to read

- values are win rates in %
- higher is better
- bold: best value per opponent
- color: relative strength

Evaluation Results (Win Rate)

Opponent	Base	LT	LT + FT
CoacAI	96.0	100.0	100.0
Mayari	67.3	100.0	100.0
WorkerRushAI	100.0	35.0	99.5
RandomBiasedAI	79.0	97.0	97.5
MixedBot	43.3	57.0	60.5
GuidedRojoA3N	74.7	87.0	88.5
Tiamat	93.0	99.0	99.0
Droplet	67.7	62.0	75.5
overall	76.5	79.2	85.7

Training opponents

- CoacAI and Mayari in LT
- WorkerRushAI in FT

Evaluation Results (Win Rate)

Opponent	Base	LT	LT + FT
CoacAI	96.0	100.0	100.0
Mayari	67.3	100.0	100.0
WorkerRushAI	100.0	35.0	99.5
RandomBiasedAI	79.0	97.0	97.5
MixedBot	43.3	57.0	60.5
GuidedRojoA3N	74.7	87.0	88.5
Tiamat	93.0	99.0	99.0
Droplet	67.7	62.0	75.5
overall	76.5	79.2	85.7

Notable MCTS gains

- MixedBot: 43.3 → 60.5
- GuidedRojoA3N: 74.7 → 88.5
- Droplet: 67.7 → 75.5

Evaluation Results (Win Rate)

Opponent	Base	LT	LT + FT
CoacAI	96.0	100.0	100.0
Mayari	67.3	100.0	100.0
WorkerRushAI	100.0	35.0	99.5
RandomBiasedAI	79.0	97.0	97.5
MixedBot	43.3	57.0	60.5
GuidedRojoA3N	74.7	87.0	88.5
Tiamat	93.0	99.0	99.0
Droplet	67.7	62.0	75.5
overall	76.5	79.2	85.7

WorkerRushAI exploit

- LT drops to 35.0%
- FT trains against WRAI
- repaired to 99.5%

Evaluation Results (Win Rate)

Opponent	Base	LT	LT + FT
CoacAI	96.0	100.0	100.0
Mayari	67.3	100.0	100.0
WorkerRushAI	100.0	35.0	99.5
RandomBiasedAI	79.0	97.0	97.5
MixedBot	43.3	57.0	60.5
GuidedRojoA3N	74.7	87.0	88.5
Tiamat	93.0	99.0	99.0
Droplet	67.7	62.0	75.5
overall	76.5	79.2	85.7

RandomBiasedAI

- LT gains remain after repair
- better against drawing behavior
- 79.0 → 97.0 after LT
- retained after FT: 97.5

Evaluation Results (Win Rate)

Opponent	Base	LT	LT + FT
CoacAI	96.0	100.0	100.0
Mayari	67.3	100.0	100.0
WorkerRushAI	100.0	35.0	99.5
RandomBiasedAI	79.0	97.0	97.5
MixedBot	43.3	57.0	60.5
GuidedRojoA3N	74.7	87.0	88.5
Tiamat	93.0	99.0	99.0
Droplet	67.7	62.0	75.5
overall	76.5	79.2	85.7

Robustness after FT

- Other examples: MixedBot, GuidedRojoA3N and Tiamat

Interpretation of the Results

Interpretation

- robustness against diverse opponents becomes part of the optimization target
- LT finds a better local optimum
- FT improves this local optimum further
- MA after LT vs Base Agent: **89%** win, **9%** draw, **2%** loss

Player 1 (P1) Priority

Effect

- P1 is prioritized in conflicts
- self-match over 200 games: P1 **38%**, draw **28%**, P2 **34%**
- visible effect, but small
- alternating the learning side prevents P1-specific behavior

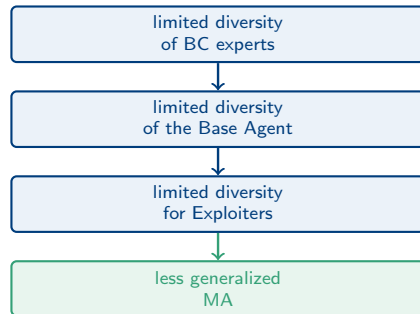
► [Open Example Video](#)

Main Limitation: Diversity

Diversity bottleneck

- not diverse exploiters discover weaknesses with similar exploits
- strategically different exploits require unlearning Base Agents fundamental behavior first
- long transitions, bad rewards
- new initial agents → more exploiter diversity

Hypothesis



► Open Example Video

Remaining Limitations and Future Work

Remaining limitations

- 1 map
- full observability, no fog of war
- high memory and computation cost
- 160 LT envs vs 24 for pure PPO

Future work

- more maps
- fog of war
- MicroRTS on GPU
- stronger diversity mechanisms

Conclusion

- LT + PPO improves robustness and generalization of the MA
- PPO fine-tuning further improves the final agent
- LT improves training and evaluation matchups
- effectiveness of LT depends on exploiter diversity
- exploiter diversity depends on initial agents and BC experts

Thank you

Questions?

Is the improvement really caused by LT, or just by more training?

- LT alone improves the overall win rate from **76.5%** to **79.2%**.
- LT + FT reaches **85.7%**.
- As a PPO-only reference, the prior BC/PPO thesis reached about **79%** and reported no improvement from further PPO training.
- The prior work also attempted a similar setup as PPO-fine-tuning, but couldn't achieve improvements to the Base Agent.

What is the strongest evidence that LT changes the policy?

- MA after LT beats the Base Agent: **89%** win, **9%** draw, **2%** loss.
- Large gains against Mayari, RandomBiasedAI, Rojo and GuidedRojoA3N.
- In the render of the evaluations the MA has at least 1 heavy unit actively blocking the opponent's Units in front of the Base.

Why does LT perform worse against WorkerRushAI?

- WorkerRushAI exposes a weakness not sufficiently found by the exploiters.
- The Base Agent is biased toward a heavy-rush strategy.
- The worker-focused initial agent for the exploiters wasn't strong enough to find it. Both of the HA created got stuck in a local optimum of drawing the Game instead.

Why does PPO fine-tuning repair it?

- WorkerRushAI is added as a direct training opponent.
- The win rate recovers from **35.0%** after LT to **99.5%**.
- Other results mostly remain stable or improve, so no strong overfitting is visible in the evaluated set.

Can we call this generalization with only one map?

- Only within the evaluated setting: one map, full observability and fixed rules.
- It is generalization across opponent types, not across maps.
- Several strong opponents were not used in training, (e. g. all MCTS-based bots).

What would be needed for a stronger claim?

- evaluation on multiple maps
- fog of war
- multiple training seeds
- same-compute PPO and ablation baselines

PPO and AlphaStar

Why use PPO instead of AlphaStar's V-trace setup?

- availability
- scope of this thesis
- V-trace would likely scale better in a large distributed setup, but is more complex.

What is different from AlphaStar?

- compute budget and scale
- PPO instead of off-policy actor-critic with V-trace
- initial policy: BC/PPO bots, not from broad human replay data

Why is exploiter diversity the main limitation?

- LT can only improve weaknesses that exploiters actually find.
- Most league agents start with the Base Agent policy.
- The Base Agent is strong, but biased toward a heavy-rush strategy.
- Very different exploits require first unlearning this default behavior to be explored.
- During this transition the exploiter performs poorly, so PPO gets weak learning signals.
- Therefore exploiters tend to find similar weaknesses instead of covering all relevant strategies.
- WorkerRushAI shows that some important weaknesses were not found by the exploiters.

Why add new initial agents?

- They give exploiters different strategic starting points.
- This improves the exploiters ability, to reach the MAs weaknesses that are only exploitable with very different strategies as the Base Agents.
- From prior runs we saw, that reaching strategies of different unit types has seemingly the biggest difficulties.
- It supports the hypothesis that BC expert diversity strongly affects LT performance.

Why keep CoacAI and Mayari during LT?

- They stabilize the early LT phase before exploiters find useful weaknesses.
- They prevent the Main Agent from drifting away from already strong behavior.
- The number of bot environments is dynamic, so they become less dominant later.

Does this make the evaluation unfair?

- CoacAI and Mayari are training opponents, so they are not the main generalization evidence.
- stronger evidence comes from opponents not used in training.
- The prior work showed that improving against CoacAI and Mayari with pure PPO is difficult.

Why adjust observations for Player 2?

- The Base Agent was trained only as Player 1.
- Gym-MicroRTS: Toward affordable full game real-time strategy games research with deep reinforcement learning implemented SP and concluded that a likely reason for worse performance of SP was that SP also has to train, to start as Player 2, while pure PPO only trained as Player 1.
- difficulties retraining the Base Agent.
- Player-2 observations, action masks and action directions are flipped.
- This lets the same policy play both sides without relearning the map orientation from scratch.

Does Player 1 priority bias the results?

- It is a potential bias in conflict resolution.
- Evaluations of the Base Agent had the same bias making the comparison fair.
- P1: LT-FT against P2: Base Agent over 100 games: win rate 85%, draw rate 15%, loss rate 0%
- P2: LT-FT against P1: Base Agent over 100 games: win rate 89%, draw rate 11%, loss rate 0%

Why use loss rate in addition to win rate?

- Draw-heavy opponents can hide defensive improvements in the win rate.
- The final loss rate drops from **7.7%** to **2.2%**.
- This suggests more robust defense, not only stronger attacks.

Are the improvements statistically reliable?

- Large gains are unlikely to be explained only by evaluation noise.
- Small differences, e. g. overall LT vs Base Agent, should be interpreted cautiously.
- most of the multiple training runs with slightly different hyperparameters showed similar improvements from LT, but the exact values varied.

Detailed Evaluation Results: Win Rates

Opponent	Main Agent without new initial agents	Base Agent	Main Agent (LT)	Main Agent + Fine-tuning
<i>bots used in training</i>				
CoacAI	100.0	96.0	100.0	100.0
Mayari	100.0	67.3	100.0	100.0
<i>scripted bots</i>				
WorkerRushAI	31.0	100.0	35.0	99.5
PassiveAI	100.0	99.7	100.0	100.0
LightRushAI	100.0	98.0	100.0	100.0
RandomAI	100.0	99.0	100.0	100.0
RandomBiasedAI	91.0	79.0	97.0	97.5
<i>MCTS based bots</i>				
Rojo	100.0	81.0	100.0	100.0
MixedBot	45.0	43.3	57.0	60.5
Izanagi	68.0	71.7	70.0	72.5
Tiamat	94.0	93.0	99.0	99.0
Droplet	45.0	67.7	62.0	75.5
GuidedRojoA3N	64.0	74.7	87.0	88.5
NaiveMCTS	0.0	0.7	2.0	6.5
overall	74.1	76.5	79.2	85.7

Detailed Evaluation Results: Loss Rates

Opponent	Main Agent without new initial agents	Base Agent	Main Agent (LT)	Main Agent + Fine-tuning
<i>bots used in training</i>				
CoacAI	0.0	2.0	0.0	0.0
Mayari	0.0	27.3	0.0	0.0
<i>scripted bots</i>				
WorkerRushAI	63.0	0.0	55.0	0.5
PassiveAI	0.0	0.0	0.0	0.0
LightRushAI	0.0	2.0	0.0	0.0
RandomAI	0.0	0.0	0.0	0.0
RandomBiasedAI	0.0	0.0	0.0	0.0
<i>MCTS based bots</i>				
Rojo	0.0	5.3	0.0	0.0
MixedBot	33.0	34.3	20.0	13.5
Izanagi	15.0	16.3	7.0	6.0
Tiamat	6.0	7.0	1.0	0.5
Droplet	22.0	9.0	22.0	10.5
GuidedRojoA3N	0.0	1.7	0.0	0.0
NaiveMCTS	0.0	2.7	0.0	0.0
overall	9.9	7.7	7.5	2.2

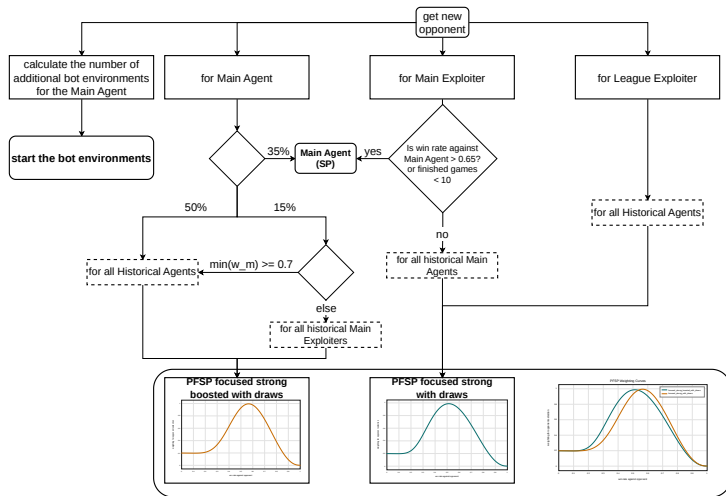
Main Agent vs League Exploiter

- the League Exploiter resets
- different hyperparameters
- historical League Exploiters are not included as Main Exploiter opponents

Why are the win rates against NaiveMCTS low?

- It is one of the few opponents that often gives up on winning and instead stays far away from the agent's units.
- This results in many draws and few losses.

Full LT Matchmaking



Matchmaking and opponent sampling in the LT setup.

New Initial Agents

1 agent per unit type

Heavy focused

- Base Agent

Light focused

- 9 epochs of BC with LightRushAI as the expert
- opponents for BC: Mayari, WorkerRushAI, LightRushAI

Worker focused

- 8 epochs of BC with WorkerRushAI as the expert
- opponents for BC: Mayari, WorkerRushAI, LightRushAI

Ranged focused

- 6 epochs of BC with CoacAI as the expert
- 180 updates of PPO against the Base Agent, CoacAI and Mayari
- opponents for BC: WorkerRushAI, LightRushAI, Mayari and CoacAI

Why Not Fewer Environments?

- The number of environments affects rollout size.
- Smaller rollouts make PPO updates more unstable because they provide less information about the update direction.
- More steps per rollout create similar tradeoffs to adding more environments.